

Midterm 2024

1. Consider the following program:

```
x = []
for i in range(2):
    x.append(i)
    print(x)
    x.append([i])
    print(x)
    x.extend([i])
    print(x)
```

1. What does the program output?
2. If we change line 7 from `x.extend([i])` to `x.append(i)`, how does that change the output and why?

2. What do the following code blocks output?

1.

```
p = [lambda: i for i in range(5)]
print([f() for f in p])
```

1.

```
p = [lambda j=i: j for i in range(5)]
print([f() for f in p])
```

1.

```
p = [lambda i=i: i for i in range(5)]
print([f() for f in p])
```

3. Write a Python function `custom_zip` that mimics the behavior of Python's built-in `zip()` function.

The function should:

- Accept a **list of lists of integers** as input.
- Return a **list of tuples**, where the first tuple contains the first elements of each list, the second tuple contains the second elements, and so on.
- **Handle lists of differing sizes** gracefully by truncating to the shortest list length

for example if your input to the function is:

```
data = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8]  
]
```

then the output of **custom_zip** should be: [(1, 4, 7), (2, 5, 8)]

4. Complete the definition for `func` below that accepts a list of lists of integers and returns a new list where its lists are ordered based on the average of their numbers. The function body should only be 1 line of code.

```
def func(x):  
    return # complete the return statement
```

```
input_list = [[3, 5, 7], [1, 2, 3], [10, 20]]  
output_list = func(input_list)
```

```
print(output_list) # Output: [[1, 2, 3], [3, 5, 7], [10, 20]]
```

Constraints: This function will naturally return the result of single expression, as corresponds to the behavior of a lambda function. You are not allowed to create and use helper functions.

5. Complete the definition for func below, which is a lambda function that returns a list of all **perfect numbers** between 1 and a given number n (inclusive).

A **perfect number** is a positive integer that is equal to the sum of its proper divisors (excluding itself but including 1).

For example:

- 6 is a perfect number because its divisors are 1, 2, 3, and (1 + 2 + 3 = 6).
- 28 is a perfect number because its divisors are 1, 2, 4, 7, 14, and (1 + 2 + 4 + 7 + 14 = 28).

Constraints: Your function body will be only one line (it will immediately return the result of some expression). You are not allowed to create and use helper functions.

```
def func():  
    return # complete the return statement
```

```
result = func(30)
```

```
print(result) # Output: [6, 28]
```

6. What do each of the following code blocks output. If there is an error, explain what the error is and why it occurs.

1.

```
a = [10, 20, 30, 40, 50]  
for i in range(len(a)):  
    del a[i]  
print(a)
```

1.

```
a = [1, 2, 3, 4, 5]  
for i in range(len(a)):  
    if a[i] % 2 == 0:  
        del a[i]  
print(a)
```

7. Consider the following Python code that defines a closure. Two instances of the closure are initialized, and each is called multiple times. What will be the output of the program?

```
def counter():
    start = 0
    def inner():
        nonlocal start
        start += 1
        return start
    return inner

counter1 = counter()
counter2 = counter()

print(counter1()) # ?
print(counter1()) # ?
print(counter2()) # ?
print(counter1()) # ?
```

8. Given a list of tuples where each tuple contains integers, write a python function whose body is just 1 line that returns a list sorted by the average of the tuples.

```
def func(x):
    return # complete the return statement
```

9. This strange syntax is possible. Fill in the blanks. What does it output and why?

```
( , ) [ ]
```

Answer:

```
("Hello", "World")[False] # outputs Hello
```

This is allowed because in python a boolean is a subclass of int